MACH ZERO · ROUND 2

Spacin' Out

Planning and understanding: fixing a Wi-Fi crash in WLED

WLED issue #5945



Our plan at a glance

THE ISSUE

WLED #5945: ESP32-C5 can crash when changing Wi-Fi band mode.

WHAT WE FOUND

In ESP-EMU, WLED crashed with a Load access fault while mDNS remained active during Wi-Fi teardown/reinit.

LIKELY CAUSE

mDNS's lwIP networking code used a Wi-Fi netif after WLED released it.

OUR FIX

Stop mDNS before Wi-Fi teardown. Let WLED's existing initInterfaces() restart it after a successful reconnect.

EMULATOR RESULT

Unfixed build: crash on cycle 2. Fixed build: 10/10 teardown/reinit cycles completed; mDNS restarted after every reconnect.

STILL NEEDED

Repeat emulator runs and validation on a real ESP32-C5 with a dual-band router.

A small change in wled00/wled.cpp · cause supported by the emulator crash and stack; hardware still to confirm

How WLED is built

Web UI	Pages you open in the browser	`wled00/data/`
Server and API	Web server, JSON, WebSocket, settings	`wled_server.cpp, json.cpp, set.cpp`
Core	The WLED class: setup() and loop()	`wled.cpp, wled.h`
LED engine	Effects, segments, LED outputs	`FX.cpp, bus_manager.cpp`
Connections	Wi-Fi, mDNS, MQTT, UDP (our issue)	`network.cpp, wled.cpp, mqtt.cpp`
Platform	Arduino-ESP32, ESP-IDF, lwIP, mDNS	`outside the repo, set in platformio.ini`

Usermods in usermods/ add extra features by hooking into the core loop.

Where things live

Where	What it holds
wled00/	Main firmware code in C++
wled00/data/	Web UI source: HTML, JS, CSS
usermods/	75 add-on modules from the community
platformio.ini	All build targets, including esp32c5dev
tools/	Build scripts and the web UI test
.github/	CI checks and the pull request template

How it builds `npm ci` `npm run build` `pio run -e esp32c5dev`

There are no C++ unit tests. CI checks that the firmware compiles for about 22 targets.

The issue we picked

What happens

On an ESP32-C5, switching the Wi-Fi band from “both” to “5 GHz” sometimes crashes the board. After a restart it works.

What the log shows

A read from an empty address. The top of the stack is mDNS code: join_group, running on the tcpip thread.

Status: reproduced in ESP-EMU

We reproduced a Load access fault in ESP-EMU while WLED was connected, mDNS was active, and WLED entered the Wi-Fi teardown/reinit path. The crash occurred on cycle 2 of a controlled 10-cycle test.

The symbolized stack matched the issue’s suspected path: mDNS lwIP code called esp_netif_is_netif_up() after the Wi-Fi netif was released.

ESP-EMU does not model the ESP32-C5’s real 5 GHz radio path, so hardware validation is still required.

Open bug, planned for WLED 17.0, reported by a project member · Source: WLED issue #5945 on GitHub

The code path to the crash

1	Band is changed in Settings	`set.cpp:140`
2	forceReconnect makes the main loop call initConnection()	`wled.cpp:980`
3	Wi-Fi is disconnected	`wled.cpp:774`
4	Wi-Fi driver is torn down and rebuilt	`wled.cpp:787`
5	mDNS is still running, then crashes	started at `wled.cpp:938`

Line numbers are from the WLED main branch we read on 6 Nov 2026.

Where it crashes

1. WLED is connected and mDNS is active.
2. User changes Wi-Fi band to 5 GHz-only.
3. WLED calls initConnection().
4. WLED disconnects Wi-Fi.
5. WLED calls WiFi.mode(WIFI_MODE_NULL).
6. The Wi-Fi netif is released.
7. mDNS lwIP code later calls esp_netif_is_netif_up() on the released netif.
8. **Load access fault.**

CRASH EVIDENCE

```
MEPC: esp_netif_is_netif_up
RA: join_group — mdns_networking_lwip.c
pcb_if_init_lwip — mdns_networking_lwip.c
tcpip_thread

MTVAL: 0x000000f3
MCAUSE: 0x5 — load access fault
```

Crash evidence from the ESP-EMU run, with a symbolized stack.

What we checked and what we did not

Checked in the code

- WLED never stops mDNS before the Wi-Fi teardown. Its only MDNS.end() is just before MDNS.begin() in initInterfaces().
- Every reconnect goes through initConnection(), so one change covers all of them.
- In the ESP-IDF mDNS source, MDNS.end() runs mdns_free(). That removes the Wi-Fi event handlers and waits for the mDNS task to stop.
- Newer mDNS code checks for an empty interface in join_group. Our crash log shows no such check.

Not checked yet

- That the crash happens on a real C5 board with a normal build. We reproduced it only in ESP-EMU.
- That the board's framework has the same mDNS code as the newest upstream version. We read upstream, not the copy pinned in platformio.ini.
- That our cause is confirmed on hardware. The emulator crash and symbolized stack support it; hardware confirmation is still needed.

mDNS files read: esp-protocols (components/mdns) and arduino-esp32 ESPmDNS.cpp, latest master

Our proposed change

Before

- Reconnect starts
- WiFi.disconnect(true)
- WiFi.mode(WIFI_MODE_NULL)
- mDNS is still active
- **Crash possible**

After

- Reconnect starts
- **NEW: if mDNS is active, call MDNS.end()**
- WiFi.disconnect(true)
- WiFi.mode(WIFI_MODE_NULL)
- Reconnect
- initInterfaces() restarts mDNS and re-registers services

Built and run in ESP-EMU; not yet tested on a real ESP32-C5.

Where the change goes

```
if (interfacesInited && strlen(cmDNS) > 0) {
  DEBUG_PRINTLN(F("[FIX] stopping mDNS before Wi-Fi teardown"));
  MDNS.end();
}
```

Place it at the top of initConnection(), before: `WiFi.disconnect(true);`

The condition is the runtime check WLED already uses; there is no compile-time flag for mDNS.

Emulator result

Unfixed build

Cycle 1: passed.

Cycle 2: crashed after Wi-Fi teardown/reinit.

Crash: Load access fault.

Stack: esp_netif_is_netif_up → join_group → tcpip_thread.

Fixed build

10/10 teardown/reinit cycles completed.

No crash, no reboot.

mDNS restarted after every reconnect.

Device remained connected afterward.

What this shows

Stopping mDNS before teardown removes the dangerous window where mDNS could use a Wi-Fi netif after WLED released it.

What this does not show

It does not prove the real 5 GHz radio path was exercised.

One 10-cycle fixed-build run so far; more repeat runs are needed.

Reading the emulator log

`mDNS started`	WLED's discovery service is active.
`[TEST] cycle N/10`	The harness confirmed WLED is connected and mDNS is active.
`[TEST] band cycle N/10 -> 5GHz only`	The harness asks WLED to enter the 5 GHz-only reconnect path.
`[FIX] stopping mDNS before Wi-Fi teardown`	The candidate fix stops mDNS while its netif still exists.
`WiFi mode_null: driver teardown / re-init.`	WLED releases and rebuilds the Wi-Fi interfaces.
`[TEST] post-teardown: mode NULL applied`	The teardown completed.
`Connected! / mDNS started`	WLED reconnected and mDNS returned with its services.
`[TEST] 10/10 teardowns done`	All planned teardown/reinit cycles completed.

From the fixed-build emulator run; N is the cycle number.

Testing strategy

Emulator validation

Done

Built a 10-cycle ESP-EMU harness that repeatedly enters the mDNS-active Wi-Fi teardown/reinit path.

Baseline: unfixed build crashed on cycle 2 with a Load access fault.

Fix: added MDNS.end() before WiFi.disconnect(). Fixed build completed 10/10 cycles without a crash. mDNS restarted after every reconnect.

Band-mode diagnostic

Run pending

Calls esp_wifi_set_band_mode() directly and logs the raw esp_err_t, band mode before and after, configured band, and channel.

Hardware validation

Pending

Requires an ESP32-C5 board and a dual-band router.

How we will validate it

Emulator evidence

Count completed teardown/reinit cycles, crashes, and mDNS restarts.

Do not count a run as successful unless:

- every cycle reaches post-teardown,
- mDNS restarts after every reconnect,
- no crash or reboot occurs.

Hardware evidence

On a real ESP32-C5:

- flash the unfixed build,
- repeat the 2.4 GHz to 5 GHz band change,
- record crashes out of attempts,
- repeat with the fix,
- verify wled.local discovery after reconnect.

Honest claim

A clean emulator run supports the fix but does not prove the hardware crash cannot happen. Hardware testing is required for that claim.

WLED asks contributors to separate “tested” from “should work”.

Completed

- Read issue #5945 and traced the crash path.
- Built an ESP-EMU reproduction harness.
- Reproduced a Load access fault on cycle 2.
- Symbolized the crash stack.
- Implemented MDNS.end() before Wi-Fi teardown.
- Built and merged a fixed ESP32-C5 emulator image.
- Completed one 10-cycle fixed-build run with no crash.
- Added a direct esp_wifi_set_band_mode() diagnostic.

Next

- Run the band-mode diagnostic.
- Repeat the fixed 10-cycle test to establish reliability; target 30 total cycles across multiple runs.
- If a C5 board becomes available, run the hardware before/after test.
- Prepare a minimal PR with only the mDNS lifecycle fix.

Pull request rules come from the WLED repo: AGENTS.md and the PR template.

Challenges so far

1. **The crash is intermittent.**
   Cycle 1 passed; cycle 2 crashed. We built a repeatable 10-cycle emulator harness instead of relying on one manual attempt.

2. **ESP-EMU does not model the C5 5 GHz radio path.**
   The emulator can run the WLED firmware and reach the teardown path, but it cannot provide a real 5 GHz access point or fully validate the 5 GHz RF transition.

3. **The Arduino Wi-Fi wrapper hides useful information.**
   WiFi.setBandMode() returns bool and can return true without calling the driver when the requested mode is already active. We call esp_wifi_set_band_mode() directly and log the raw esp_err_t.

Challenges so far, continued

4 **Our first fix used a nonexistent compile-time flag.**

WLED_ENABLE_MDNS does not exist in the repository. We replaced it with the runtime condition actually used by WLED: interfacesInited && strlen(cmDNS) > 0.

5 **A simple mDNS restart would reduce discovery quality.**

initInterfaces() already restarts mDNS and re-registers _http._tcp, _wled._tcp, and the MAC TXT record. We let it do that instead of calling MDNS.begin() manually.

6 **One clean fixed run is not proof.**

The unfixed build crashed once in two cycles; the fixed build passed 10 cycles once. More repeat runs are needed before claiming the fix reliably prevents the race.